

HW02 — Domain Testing on EShop: Main Report

Exercise	HW02-AI — Domain Testing on EShop
Student	Lê Thiên Phú (23127244)
Fork under test	https://github.com/lamTpG/eshop-sut
AI tool used	Claude Code (Claude Sonnet 5 / Opus 4.8) — see AI Audit Report appendix
Techniques	Equivalence Partitioning (EP), Boundary Value Analysis (BVA), combinatorial selection

This is the umbrella report. Each feature has a full, standalone Domain Testing + BVA report with the complete step-by-step derivation, variable matrices, partition maps, boundary tables, and per-test-case execution results:

- FR-04 — Personal Profile Management
- FR-08 — Checkout
- FR-19 — User Management (Admin)
- FR-11 — Order History View (Mobile)

1. System Under Test & feature selection

SUT: EShop — a Vietnamese e-commerce demo application (React web + React admin + React Native mobile + Express/SQLite backend), pulled from the instructor repository and forked.

Four features were selected, one per pool, per the assignment's no-duplication rule:

Pool	Feature	Surface
A	FR-04 — Personal Profile Management	Web (PUT /api/users/me)
B	FR-08 — Checkout	Web (POST /api/checkout)
C	FR-19 — User Management (Admin)	Admin panel (GET/DELETE /api/admin/users)
D	FR-11 — Order History View	Mobile (GET /api/orders/my-orders, /orders/:id)

2. Methodology

Testing was driven by a custom Claude Code **domain-testing Agent Skill** built for this assignment (submitted separately with a demo video). The skill enforces a strict, human-in-the-loop 6-step workflow, and the AI was used as a *disciplined assistant* walked through every step — never as a single black-box "find the bugs" prompt.

Step	Activity
0	Pre-requisite validation — read the actual source (frontend, backend route, DB schema), compare code against the instructor's spec, elicit ambiguities

Step	Activity
1	Variable hunt — enumerate explicit and implicit input/output/state variables
2	Domain + Equivalence Partitioning — define each variable's domain, split into valid/invalid classes
3	Boundary Value Analysis — LB/LB±1, UB/UB±1 on ordered domains
4	Multi-variable dependencies & combinatorial strategy (Pairwise / Exhaustive / Isolated-Boundaries + Happy-Path)
5	Test-case generation with selection optimization (combine valid classes, isolate invalid classes)

After design, every **Direct-API** test case was executed for real via `curl` against a freshly-seeded backend, and every **UI/Mobile** test case was executed manually in the browser / Expo app. Each report's "Actual / Bug flag" column records observed results, not predictions.

3. Results summary

Feature	Test cases designed	Channel split	Distinct confirmed defects
FR-04 Profile	37	API + UI	5
FR-08 Checkout	20	API + UI	4
FR-19 Admin Users	15	API + UI	3
FR-11 Order History	11	API + UI	2
Total	83		14 in-scope + 2 bonus

The 14 in-scope defects are each filed as a GitHub issue (see [github-issues/](#)). Test-case pass/fail/not-executed counts are in [HW02-README.md](#), derived from each report's per-case "Actual" column.

4. Consolidated bug inventory

Severity: **CRIT** = critical (security or spec-core violation), **MAJ** = major functional defect, **MIN** = minor / contract inconsistency.

FR-04 — Profile Management

#	Sev	Defect	Test cases	Screenshot	Issue
1	CRIT	Role privilege escalation — <code>PUT /api/users/me</code> writes a client-supplied <code>role</code> , letting any user self-promote to <code>admin</code>	TC-31, TC-33	FR-04-TC-31-1/2	BUG-A-10#32
2	MAJ	Phone regex mismatch — frontend regex rejects spec-valid numbers (start-0, 10–11 digits) and accepts spec-invalid ones (wrong prefix)	TC-11/13/15/23	FR-04-TC-11/13/15/23	BUG-A-11#33

#	Sev	Defect	Test cases	Screenshot	Issue
3	MAJ	Empty phone blocks unrelated saves — optional phone left empty blocks saving name/address	TC-11	FR-04-TC-11	BUG-A-12 #34
4	MAJ	Zero server-side validation on name/phone/address (empty, whitespace, oversized, malformed all persisted)	TC-05–07, 17–21	—	BUG-A-13 #35
5	MIN	Silent NULL-coercion on field omission — omitting a field wipes it to NULL instead of leaving it unchanged	TC-08/10/22	FR-04-TC-08-1/2	BUG-A-14 #36

FR-08 — Checkout

#	Sev	Defect	Test cases	Screenshot	Issue
1	CRIT	Backend never recalculates <code>total_amount</code> — client value persisted verbatim (spec explicitly forbids)	TC-05–15, 17	FR-08-TC-01-editable-total	BUG-B-08 #38
2	CRIT	Checkout total directly editable in UI — plain editable <code><input></code> , opposite of spec	TC-01, 05–15	FR-08-TC-01-editable-total	BUG-B-07 #37
3	MAJ	Cart never cleared after successful checkout	TC-01	FR-08-TC-01-cart-not-cleared-1/2	BUG-B-09 #39
4	MAJ	No cart-size guard — order created for an empty cart; stacks with #1 into a fully-priced phantom order	TC-16/17	—	BUG-B-10 #40
5	MIN	<code>total_amount</code> accepts negative / zero / decimal / null (part of #1's root cause)	TC-07–09, 14/15	—	same as #1

FR-19 — User Management (Admin)

#	Sev	Defect	Test cases	Screenshot	Issue
1	CRIT	No role check on admin endpoints — any authenticated user (any role string) can list/delete users, incl. the sole admin	TC-04/05/12/13/15	FR-19-TC-04, FR-19-TC-15-1/2	BUG-C-08 #41
2	CRIT	No self-deletion guard — admin can delete own logged-in account, no confirmation dialog	TC-07/14	FR-19-TC-14-1/2	BUG-C-09 #42
3	MIN	DELETE ignores <code>this.changes</code> — non-existent / malformed ids return false-positive <code>200 "User deleted"</code>	TC-08–11	FR-19-TC-11	BUG-C-10 #43

FR-11 — Order History (Mobile)

#	Sev	Defect	Test cases	Screenshot	Issue
1	CRIT	IDOR on GET /api/orders/:id — no auth, no ownership filter; any caller reads any user's order	TC-10	FR-11-TC-10	BUG-D-10 #44
2	MAJ	No status color differentiation — spec requires color-coded statuses; all render identical	TC-11	FR-11-TC-11	BUG-D-11 #45

Bonus findings (outside the 4 assigned features' scope, surfaced during testing)

Sev	Defect	How found	Owning FR
CRIT	SAVE10 coupon math inversion — percent formula treats <code>discount_value</code> (10) as a whole number, so a "10% off" coupon multiplies the total $\sim 10\times$ (58,000,000 $\rightarrow$ 580,000,000)	Execution of FR-08 TC-18	FR-09
MAJ	Mobile order-history staleness — admin status changes never refresh the user's history screen until they cancel/place an order	Exploratory testing	FR-11/FR-20 (sync)

5. AI Gap Analysis

Per requirement §6.3: below are the concrete cases where the AI-driven process missed, mis-scored, or could not resolve something on its own — and *why*, categorized as **prompt/input quality**, **AI tool limitation**, or **inherent technique/complexity**.

Gap 1 — False positive: over-flagged the checkout "route guard" (prompt/interpretation)

What happened. For FR-08, the AI reported "no route guard on `/checkout`" as a bug: a logged-out user can open the checkout page by URL. Manual testing plus a careful re-reading of the literal spec showed this is **not** a defect — the spec says "*chỉ người dùng đã đăng nhập mới tiến hành thanh toán được*" (only logged-in users may **proceed to checkout/payment**), which constrains the checkout *action*, not page visibility. The action is correctly blocked (the API returns 401 for any guest payment attempt).

Why the AI missed it. Prompt/interpretation bias. The AI over-interpreted the requirement, reading an implied "the page must be inaccessible" security expectation into text that only constrained the payment action. LLMs are biased toward flagging anything that "looks insecure," even when it isn't spec-mandated. The correction came from human domain judgment and literal-spec re-reading — not from the model. **Lesson:** the human must hold the AI to the *literal* requirement and reject inferred requirements the spec never states.

Gap 2 — Complete miss: mobile order-history staleness (inherent technique limitation)

What happened. When an admin changes an order's status, the mobile user's order-history screen does not refresh until the user cancels or places another order. Domain Testing + BVA never surfaced this; it was found by exploratory ("just using the app") testing.

Why the AI missed it. This is an inherent limitation of the *technique*, not a failure of the model. EP and BVA operate on the domain of **input values** — they ask "for input X in class P, is the behavior correct?" This defect has no offending input value at all; it is a **temporal, cross-actor state-synchronization gap** between two independent clients (admin panel and mobile app) over time. No input partition or boundary could ever target it. It belongs to state-transition testing or exploratory/session-based testing. **Lesson:** domain testing is powerful for input-driven correctness but structurally blind to timing/state-sync bugs; a complete test strategy must combine techniques.

Gap 3 — Wrong static assumption caught only by execution: **SAVE10** coupon math (scoping + tool limitation)

What happened. During FR-08 *design*, the AI assumed applying **SAVE10** (10% off) would produce `final_amount = 52,200,000`. **Execution** revealed the real result is `580,000,000` — the coupon arithmetic is inverted (`total × (1 - discount_value)` with `discount_value = 10`, not `0.10`).

Why the AI missed it. Two compounding causes. (a) **Scoping/prompt:** coupon internals were explicitly out of scope (owned by FR-09), so the Step-0 code-grounding never read the coupon-calculation code — the AI reasoned from the spec's *intent* ("10% off") and assumed correctness. (b) **Tool limitation:** static reasoning cannot catch an arithmetic bug it never looked at; only running the endpoint exposed it. **Lesson:** predicted results from static code-reading are hypotheses — actually executing the tests is what converts them to facts, and dynamic testing routinely catches what static reasoning assumes away.

Gap 4 — Could not resolve runtime behavior alone: field-omission NULL coercion (tool limitation)

What happened. The AI correctly *designed* the field-omission cases (FR-04 TC-08/10/22, FR-08 TC-15) but had to mark their expected results "**undetermined pending execution**" — it could not predict whether the Node `sqlite3` driver would bind an omitted (`undefined`) parameter as SQL `NULL`, throw, or leave the column unchanged. Execution resolved it: the driver silently coerces to `NULL`, wiping the field.

Why the AI missed it. Tool limitation. A static-analysis model cannot reliably determine runtime, library-version-specific driver behavior without executing. Notably, the AI handled this *well* — it flagged the uncertainty honestly instead of guessing a confident-but-possibly-wrong answer. **Lesson:** a trustworthy AI assistant should mark what it cannot know rather than fabricate certainty; execution then closes the gap.

Gap 5 — Anchoring on the wrong oracle without the human's spec (prompt/input quality)

What happened. The AI's code inspection found the frontend phone regex `/^[1-9][0-9]{8,9}$/`. Left to the code alone, it would have treated that implementation as the correct oracle. The authoritative rule — "*starts with 0, 10–11 digits*" — came from the **human supplying the instructor's Vietnamese spec**, which contradicts the code. Only with that spec did the regex become a *bug* rather than the standard.

Why the AI missed it (would have). Prompt/input quality. The AI grounds itself in the code plus whatever spec it is given; when the code contradicts the true requirement and that requirement is not in the prompt, the AI anchors on the implemented behavior. The human's domain knowledge (Vietnamese phone format) and possession of the real spec were essential to setting the correct oracle. **Lesson:** the AI's output is only as correct as the requirements it is given — the human owns supplying and validating the source of truth.

Cross-cutting reliability note

During API execution one sub-session self-reported "27 test cases run" when 32 had actually been executed (a counting slip in its own summary, while the underlying work was correct and complete). This reinforces that **every AI claim was independently verified** against the actual report files and code before being accepted — the human review loop is not optional.

6. Scope discipline

Two findings were deliberately kept **out** of the assigned features' defect counts and are recorded as bonus notes only: the **SAVE10** coupon-math bug (owned by FR-09) and the order-history staleness bug (a sync concern spanning FR-11/FR-20). Conversely, one initially-reported FR-08 finding (the checkout "route guard") was **retracted** after manual verification showed it did not violate the literal spec. Maintaining these boundaries — reporting real bugs in-scope, flagging out-of-scope ones separately, and retracting false positives — was a deliberate part of the process.

Appendices — AI Critique (§10) and AI Audit Report (§9) — are provided as separate documents per the submission structure.

FR-04 — Personal Profile Management: Domain Testing / BVA Test Case Suite

Methodology: Domain Testing via Equivalence Partitioning (EP) and Boundary Value Analysis (BVA), following a 7-step human-in-the-loop QA process. Combinatorial strategy: **Isolated Boundaries + Happy-Path Interactions** (Option C), with deliberate targeted channel crossings for Phone Number and API-only injection tests for Email/Role.

Source spec (FR-04, Vietnamese, instructor-provided):

Người dùng đã đăng nhập có thể cập nhật: Họ Tên, Số điện thoại, Địa chỉ giao hàng mặc định. Số điện thoại hợp lệ: bắt đầu bằng số 0, từ 10–11 chữ số. Email không được phép thay đổi qua giao diện. Người dùng chỉ có thể cập nhật hồ sơ của chính mình; không thể tự thay đổi thuộc tính role.

Code under test:

- Frontend: `frontend-web/src/pages/Profile.jsx`
- Backend: `backend/server.js` (GET/PUT `/api/users/me`)
- DB schema: `backend/database.js` (`users` table)
- Session state: `frontend-web/src/context/AuthContext.jsx`

Preconditions (unless a case states otherwise): Actor is logged in as a standard `role='user'` account, holding a valid JWT, submitting via `PUT /api/users/me`.

Assumption Legend

Tag	Assumption
A1	Name business cap = 100 chars (assumed; not enforced by spec or code) — Step 2
A2	Shipping address cap = 255 chars (assumed; not enforced by spec or code) — Step 2
A3	Whitespace-only name is invalid by business rule — Step 2 Flag A
A4	Phone is strict digits-only, no separators/country code — Step 2 Flag B
A5	Phone is optional; empty string is valid — Step 2 Flag C
A6	Written spec (not the current frontend regex) is the source of truth for phone's valid domain — Step 0 decision #1
A7	Filler value for a field NOT under test is chosen to avoid masking the field that IS under test
A8	No phone value is simultaneously spec-valid AND accepted by today's buggy frontend regex — a frontend-passing but spec-noncompliant placeholder is used for UI happy-path confirmation, and flagged as a deliberate deviation
A9	Auth-state cases use one fixed nominal valid payload; not crossed with field boundaries — Step 5

Tag	Assumption
A10	Cross-user case is a single confirmatory regression guard, not a boundary sweep — Step 0 decision #6
A11	Field-omission (JSON key entirely absent) tested via Direct API only — Step 3 logic-check #1
A12	Email/Role injection cases pair the injected field with an otherwise-valid Name/Phone/Address baseline, so the injected field is the sole variable under test
A13	The combined Email+Role "kitchen-sink" case is an added-value scenario beyond core EP methodology — Step 3 logic-check #3

Nominal valid fillers used throughout: **Name** = "Nguyen Van A", **Phone** = "" (empty, spec-valid per A5), **Address** = "123 Nguyen Trai, Thanh Xuan, Hanoi".

1. Full Name — combined valid boundary sweep (Name × Address paired to reduce bloat, per Step 6 optimization rule)

Channel: Direct API. Phone filler = "" [A5, A7].

ID	Name value	Address value	Expected (per spec)	Actual / Bug flag	Assumptions
TC-01	"A" (1, LB)	"" (0, LB)	200, both fields saved	Executed: 200, name : "A", shipping_address : "" persisted exactly as sent (verified via GET). Matches — no defect	A7
TC-02	"An" (2, LB+1)	1-char string (LB+1)	200, both fields saved	Executed: 200, name : "An", shipping_address : "X" persisted. Matches — no defect	A7
TC-03	99-char string (UB-1)	254-char string (UB-1)	200, both fields saved	Executed: 200, full 99-char name and 254-char address persisted verbatim. Matches — no defect	A7
TC-04	100-char string (UB)	255-char string (UB)	200, both fields saved	Executed: 200, full 100-char name and 255-char address persisted verbatim. Matches — no defect	A1, A2, A7

2. Full Name — isolated invalid classes

Channel: Direct API. Phone/Address filler = nominal valid values [A7].

ID	Name value	Expected (per spec)	Actual / Bug flag	Assumptions
TC-05	"" (empty)	400 — rejected	Executed: 200, name : "" persisted. Bug confirmed: backend has zero validation.	A7

ID	Name value	Expected (per spec)	Actual / Bug flag	Assumptions
TC-06	" " (whitespace-only)	400 — rejected	Executed: 200, name : " " persisted verbatim. Bug confirmed : same as above.	A3, A7
TC-07	101-char string (UB+1)	400 — rejected (cap violation)	Executed: 200, full 101-char value persisted. Bug confirmed : no cap enforced.	A1, A7
TC-08	name key omitted from JSON body	Undetermined pending execution — this endpoint has no defined behavior for an omitted required field; will be resolved to a concrete Pass/Fail once executed and the actual behavior is documented.	Executed: 200, no crash. The omitted name key was bound as undefined , which node's sqlite3 driver coerced to SQL NULL — GET /api/users/me afterward showed name: null , other fields untouched. Resolves the prior "undetermined" prediction: this driver version silently NULLs rather than throwing. Bug : an omitted field silently wipes existing data instead of leaving it unchanged or being rejected.	A11

3. Shipping Address — isolated invalid classes

Channel: Direct API. Name/Phone filler = nominal valid values [A7].

ID	Address value	Expected (per spec)	Actual / Bug flag	Assumptions
TC-09	256-char string (UB+1)	400 — rejected (cap violation)	Executed: 200, full 256-char value persisted. Bug confirmed : no cap enforced.	A2, A7
TC-10	shipping_address key omitted from JSON body	Undetermined pending execution — this endpoint has no defined behavior for an omitted required field; will be resolved to a concrete Pass/Fail once executed and the actual behavior is documented.	Executed: 200, no crash. Omitted shipping_address key bound as undefined → coerced to NULL ; GET confirmed shipping_address: null . Resolves the prior "undetermined" prediction, consistent with TC-08. Bug : omission silently wipes the field.	A11

4. Phone Number — valid/optional classes, dual-channel (core bug investigation)

Name/Address filler = nominal valid values [A7].

ID	Phone value	Channel	Expected (per spec)	Actual / Bug flag	Assumptions
TC-11	"" (empty, V2)	UI	200, phone cleared, other fields saved	Executed (manual UI test): submission blocked entirely by a client-side alert ; name/address changes were also not saved , since the whole form submission is blocked before any request is sent. Bug confirmed: current frontend regex wrongly rejects empty phone input, and this also prevents saving unrelated fields — confirms Flag C bug	A5, A6, A8
TC-12	"" (empty, V2)	API	200, phone cleared	Executed: 200, phone: "" persisted. Matches spec — no defect via API	A5
TC-13	"0912345678" (10, LB)	UI	200, phone saved	Executed (manual UI test): submission blocked entirely by a client-side alert ; name/address changes were also not saved . Bug confirmed: frontend regex requires first digit 1–9; this spec-valid value is wrongly rejected client-side, also blocking unrelated field saves	A6, A8
TC-14	"0912345678" (10, LB)	API	200, phone saved	Executed: 200, phone: "0912345678" persisted. Matches spec via API, but only because backend has no validation at all (accepted for the wrong reason)	A6
TC-15	"09123456789" (11, UB)	UI	200, phone saved	Executed (manual UI test): submission blocked entirely by a client-side alert ; name/address changes were also not saved . Bug confirmed: rejected client-side, same regex mismatch as TC-13, also blocking unrelated field saves	A6, A8
TC-16	"09123456789" (11, UB)	API	200, phone saved	Executed: 200, phone: "09123456789" persisted. Matches spec via API (again, absence of validation, not correctness)	A6

5. Phone Number — isolated invalid classes

Channel: Direct API (primary — exposes total absence of server-side validation). Name/Address filler = nominal valid.

ID	Phone value	Expected (per spec)	Actual / Bug flag	Assumptions
TC-17	"0" (1 digit)	400 — rejected (too short)	Executed: 200, phone: "0" persisted as-is. Bug confirmed.	A4, A6
TC-18	"091234567" (9 digits, LB-1)	400 — rejected (too short)	Executed: 200, phone: "091234567" persisted as-is. Bug confirmed.	A6
TC-19	"091234567890" (12 digits, UB+1)	400 — rejected (too long)	Executed: 200, phone: "091234567890" persisted as-is. Bug confirmed.	A6
TC-20	"1912345678" (10 digits, wrong prefix)	400 — rejected (must start with 0)	Executed: 200, phone: "1912345678" persisted as-is. Bug confirmed.	A6
TC-21	"091-2345678" (non-digit char)	400 — rejected (charset)	Executed: 200, phone: "091-2345678" persisted as-is (hyphen retained). Bug confirmed.	A4
TC-22	phone key omitted from JSON body	Undetermined pending execution — this endpoint has no defined behavior for an omitted required field; will be resolved to a concrete Pass/Fail once executed and the actual behavior is documented.	Executed: 200, no crash. Omitted phone key bound as undefined → coerced to NULL ; GET confirmed phone: null . Resolves the prior "undetermined" prediction, consistent with TC-08/TC-10. Bug: omission silently wipes the field.	A5, A11

6. Phone Number — wrong-prefix class via UI (extra channel crossing)

ID	Phone value	Channel	Expected (per spec)	Actual / Bug flag	Assumptions
TC-23	"1912345678" (10 digits, wrong prefix)	UI	Rejected client-side (spec-invalid)	Executed (manual UI test): submission succeeded, all fields saved. Confirmed bug: current frontend regex [1-9][0-9]{8,9} actively accepts this value and submits it — the exact inverse of the spec rule	A6, A8

7. Email — immutability control (API only; UI cannot send this field)

Name/Phone/Address = nominal valid baseline [A12].

ID	Email in payload	Expected	Actual / Bug flag
TC-24	Absent	Email unchanged, 200	Executed: 200, email remained <code>test@eshop.com</code> . Matches — no defect
TC-25	Same as current value	Email unchanged, 200	Executed: 200, email remained <code>test@eshop.com</code> . Matches — no defect
TC-26	Different valid email	Email unchanged, 200	Executed: 200, email remained <code>test@eshop.com</code> despite <code>email:"new@example.com"</code> in the payload. Matches — no defect (email not in UPDATE query at all)
TC-27	Malformed value (<code>"notanemail"</code>)	Email unchanged, 200	Executed: 200, email remained <code>test@eshop.com</code> despite the malformed value in the payload. Matches — no defect

8. Role — privilege-escalation investigation (API only; UI cannot send this field)

Name/Phone/Address = nominal valid baseline [A12].

ID	Role in payload	Backend code path	Expected (per spec)	Actual / Bug flag
TC-28	Absent	Falsy → skipped	Role unchanged, 200	Executed: 200, role remained <code>"user"</code> . Matches — no defect
TC-29	<code>""</code> (empty string)	Falsy → skipped	Role unchanged, 200	Executed: 200, role remained <code>"user"</code> . Matches — no defect (safe by JS truthiness accident, not by design)
TC-30	<code>"user"</code> (current value)	Truthy → vulnerable UPDATE runs	Role unchanged, 200	Executed: 200, role remained <code>"user"</code> (self-value re-applied via the truthy branch). No observable defect for this input — but confirms the vulnerable code path executes unconditionally whenever role is truthy, corroborating the TC-31/TC-33 escalation finding rather than standing as its own defect.
TC-31	<code>"admin"</code>	Truthy → vulnerable UPDATE runs	Role unchanged, 200	Executed: 200; <code>GET /api/users/me</code> confirmed role was overwritten from <code>"user"</code> to <code>"admin"</code> . Confirmed critical security bug — privilege escalation. (Role reset back to <code>"user"</code> immediately after, for downstream test cleanliness.)
TC-32	<code>"xyz"</code> (garbage)	Truthy → vulnerable	Role unchanged,	Executed: 200; GET confirmed role overwritten to <code>"xyz"</code> . Confirmed bug: role overwritten to an

ID	Role in payload	Backend code path	Expected (per spec)	Actual / Bug flag
		UPDATE runs	200	undefined/invalid role value. (Reset back to "user" after.)

9. Combined kitchen-sink attack (added-value case, beyond core EP methodology) [A13]

ID	Payload	Channel	Expected	Actual / Bug flag
TC-33	Valid Name/Phone/Address + email="attacker@evil.com" + role="admin"	API	Email and role both unchanged, 200	Executed: 200; GET confirmed email stayed test@eshop.com (safe), but role was overwritten to "admin" — the escalation succeeds even hidden inside an otherwise-ordinary profile update, with no suspicious-looking request shape. Confirmed. (Role reset back to "user" after.)

10. Authentication State (fixed regression-guard set) [A9]

ID	Auth condition	Payload	Expected	Actual / Bug flag
TC-34	Missing Authorization header	Nominal valid	401	Executed: request with no Authorization header returned 401 {"error":"Unauthorized"}. Matches — no defect
TC-35	Invalid/expired token	Nominal valid	403	Executed: request with a malformed JWT (Bearer invalid.token.value) returned 403 {"error":"Forbidden"}. Matches — no defect

11. Cross-user regression guard (single confirmatory case) [A10]

ID	Scenario	Expected	Actual / Bug flag
TC-36	User A holds a valid token, submits a profile update; User B's row is checked afterward	Only User A's row is modified; User B's row is untouched	Executed: logged in as Admin (User A) and Test User (User B) separately; Admin submitted a profile update (name/phone/address); GET afterward confirmed Test User's row was completely untouched, and only Admin's own row changed. Matches — structurally guaranteed (row target is always req.user.id from the JWT; no client-suppliable target-user field exists on this endpoint). (Admin's profile fields were reverted to seed values after the test.)

12. UI channel — current-implementation happy path (confirmatory) [A8]

ID	Name	Phone	Address	Channel	Expected (given today's buggy frontend)	Note
TC-37	"Nguyen Van A"	"912345678" (frontend-passing, spec-noncompliant)	"123 Nguyen Trai, Thanh Xuan, Hanoi"	UI	200, all three fields saved	Executed (manual UI test): submission succeeded, all three fields saved as expected. This is not the spec-ideal happy path — no phone value can satisfy both the spec and today's frontend regex simultaneously (see TC-11/13/15/23). This case only confirms the save mechanism works end-to-end today.

Summary of Confirmed/Expected Defects

- Phone regex mismatch (frontend vs. spec)** — `Profile.jsx` line 43 requires first digit 1–9 and 9–10 total digits; spec requires first digit 0 and 10–11 total digits. Every spec-valid phone number is rejected by the UI (TC-11, TC-13, TC-15); the wrong-prefix class is wrongly accepted (TC-23). **GitHub Issue:** [BUG-A-11 #33](#)
- Empty phone blocks unrelated saves** — the buggy regex rejects "", so a user cannot save just their name/address without a valid phone on file (TC-11), even though phone should be optional (Flag C). **GitHub Issue:** [BUG-A-12 #34](#)
- Zero server-side validation** on `name`, `phone`, `shipping_address` — every length/format/charset invalid class is silently accepted and persisted by the backend (TC-05–TC-07, TC-17–TC-21). **GitHub Issue:** [BUG-A-13 #35](#)
- Critical: role privilege-escalation vulnerability** — `PUT /api/users/me` applies a client-supplied `role` field whenever it's truthy (`server.js` lines 118–129), letting any authenticated user set their own `role` to "admin" (TC-31), including hidden inside an otherwise-normal profile update (TC-33). **GitHub Issue:** [BUG-A-10 #32](#)
- No confirmed defect** in: email immutability, auth-state gating, or cross-user isolation.
- Confirmed: silent NULL-coercion on field omission** — when `name`, `shipping_address`, or `phone` is omitted entirely from the JSON payload (TC-08, TC-10, TC-22), the endpoint returns 200 with no crash; the omitted key is bound as `undefined` and node's sqlite3 driver coerces it to SQL `NULL`, silently wiping the existing value instead of leaving it unchanged or rejecting the request. A partial-update client that omits an untouched field will unintentionally erase it. **GitHub Issue:** [BUG-A-14 #36](#)

FR-08 — Checkout: Domain Testing / BVA Test Case Suite

Methodology: Domain Testing via Equivalence Partitioning (EP) and Boundary Value Analysis (BVA), following a 6-step human-in-the-loop QA process. Combinatorial strategy: **Isolated Boundaries + Happy-Path Interactions** (Option C), with three deliberate targeted interactions (empty-cart × large-tampered-total stacking, a coupon smoke test, and a full happy-path combining product-list/address/cart-clear assertions). 20 test cases total.

Source spec (FR-08, Vietnamese, instructor-provided):

Chỉ người dùng đã đăng nhập mới tiến hành thanh toán được. Tổng tiền thanh toán được tính tự động từ giỏ hàng và không cho phép người dùng chỉnh sửa trực tiếp. Giao diện hiển thị đầy đủ danh sách sản phẩm đặt mua. Backend phải tự tính lại tổng tiền; không chấp nhận giá trị total_amount do client gửi lên. Sau thanh toán thành công, giỏ hàng được xóa.

Code under test:

- Frontend: frontend-web/src/pages/Checkout.jsx, frontend-web/src/pages/Cart.jsx
- Cart state: frontend-web/src/context/CartContext.jsx
- Backend route: backend/server.js (POST /api/checkout, authenticateToken, POST /api/apply-coupon, POST /api/coupon-usage)
- DB schema: backend/database.js (orders, products tables)
- Routing: frontend-web/src/App.jsx

Preconditions (unless a case states otherwise): Actor is logged in as a standard role='user' account, holding a valid JWT, submitting via POST /api/checkout.

Fixture cart (used throughout, drawn from actual database.js seed data):

Item	Unit Price (VND)	Qty	Subtotal
iPhone 15 Pro Max	30,000,000	1	30,000,000
Samsung Galaxy S24 Ultra	28,000,000	1	28,000,000
True Cart Total (TT)			58,000,000

Assumption Legend

Tag	Assumption
A1	Fixture cart = iPhone 15 Pro Max (30,000,000) ×1 + Samsung Galaxy S24 Ultra (28,000,000) ×1 = True Total (TT) 58,000,000 VND, drawn from real seed data — Step 3
A2	Numeric total_amount tamper values restricted to non-negative integers; decimal/float values folded into the malformed (I5) class rather than a separate BVA sweep — Step 2 ambiguity #2

Tag	Assumption
A3	Upper business-cap boundary set at 1,000,000,000 VND as a realistic-but-implausible probe, chosen over an architectural max as the primary upper boundary — Step 2 ambiguity #3
A4	Architectural-extreme probe (<code>Number.MAX_SAFE_INTEGER</code>) is an added-value case beyond core BVA methodology, mirroring FR-04's kitchen-sink treatment — Step 3
A5	Empty-cart checkout submission is treated as a confirmed defect (not merely an untested edge), per explicit scoping decision — Step 2 ambiguity #1
A6	Coupon feature is included only as a single binary variable (applied / not applied); internal coupon business rules (discount math, expiry, usage caps) are explicitly out of scope, owned by FR-09 — Step 0 decision #2
A7	<code>shipping_address</code> receives one lightweight confirmatory case only, not a full BVA sweep, since it is absent from the FR-08 spec and overlaps FR-04's own address boundary analysis — Step 0 decision #6
A8	Post-purchase display of purchased products (order history/detail) is explicitly out of scope for FR-08 — the missing <code>order_items</code> persistence is a structural gap relevant to FR-11 (order history), not a testable FR-08 behavior — Step 0 decision #4
A9	Cart quantity / line-item-count bounds are out of scope; the fixture cart is used as-is across all cases, not swept — Step 0 decision #5
A10	Auth-state invalid classes (missing/invalid token) are tested in isolation with an otherwise-honest body, since <code>authenticateToken</code> short-circuits before any other field is read — Step 4 finding #1
A11	Each <code>total_amount</code> tampering test case pairs a single invalid class with an otherwise-honest cart/token/address, per the invalid-class isolation rule — Step 5 selection rule

1. Full happy-path (all valid classes combined, per Step 5 optimization rule)

Channel: Direct API. Token = valid; Cart = fixture (A1); Coupon = not applied; `shipping_address` = non-empty valid string.

ID	<code>total_amount</code>	Expected (per spec)	Actual / Bug flag	Assumptions
TC-01	<code>58000000</code> (= TT, honest)	200; order persisted with recalculated total = 58,000,000; full product list shown pre-payment; cart cleared after success	Executed: 200, <code>orderId:3</code> created with <code>total_amount</code> persisted = 58,000,000 (confirmed via <code>GET /api/orders/my-orders</code>) — but only because the honest value happens to match, not because the backend recalculates anything (server.js:297-309 inserts <code>req.body.total_amount</code> verbatim). Bug confirmed: <code>GET /api/cart</code> immediately after checkout still returned both fixture items — <code>clearCart()</code> is never invoked; cart remains	A1

ID	<code>total_amount</code>	Expected (per spec)	Actual / Bug flag	Assumptions
			populated after a successful order. Additionally manually verified end-to-end via the real UI (all other <code>total_amount</code> cases in this report, TC-01/05-15, were run via Direct API only): editing the total in the actual Checkout page input and submitting produced an order persisted with the tampered value, confirming defects #1 and #2 together work through the real browser, not just via direct API calls	

2. Authentication — isolated invalid classes [A10]

Channel: Direct API unless noted. Otherwise-honest body (fixture cart, `total_amount` = 58000000, valid address).

ID	Auth condition	Expected (per spec)	Actual / Bug flag	Assumptions
TC-02	Missing <code>Authorization</code> header	401; no order created	Executed: 401 { <code>"error": "Unauthorized"</code> }, no order created. Matches — <code>server.js:103</code> returns 401 before any other field is read. No defect	A10
TC-03	Invalid/malformed token signature	Rejected; no order created	Executed: 403 { <code>"error": "Forbidden"</code> }, no order created. Matches — <code>server.js:106</code> correctly rejects the request. No defect	A10
TC-04	UI: direct navigation to <code>/checkout</code> by URL while not logged in	The spec requires only that a guest cannot actually complete a checkout/payment ("tiến hành thanh toán được") — not that the checkout page itself be inaccessible to view	No defect (corrected — originally over-flagged as a bug). <code>App.jsx:58</code> registers <code>/checkout</code> with no page-level auth guard, and <code>Checkout.jsx</code> renders regardless of login state. But the spec's literal requirement concerns the checkout <i>action</i> , not page visibility: any attempt to actually submit a payment as a guest still correctly fails at the API (401, per TC-02), so the business rule is enforced. A guest viewing a non-functional checkout form is a UX/defense-in-depth observation at most, not a violation	—

ID	Auth condition	Expected (per spec)	Actual / Bug flag	Assumptions
			of this spec line. Manually verified: guests can view the cart, but only logged-in users can proceed through to a working checkout	

3. `total_amount` — honest-value boundary (near-TT tampering) [A2, A11]

Channel: Direct API. Valid token; fixture cart; no coupon; valid address.

ID	<code>total_amount</code>	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-05	57999999	TT-1	Recalculated to 58,000,000	Executed: 200, <code>orderId:4</code> , <code>total_amount</code> persisted verbatim as 57,999,999 (confirmed via GET). Bug confirmed: backend performs zero recalculation, even for a 1-VND tamper	A2, A11
TC-06	58000001	TT+1	Recalculated to 58,000,000	Executed: 200, <code>orderId:5</code> , <code>total_amount</code> persisted verbatim as 58,000,001 (confirmed via GET). Bug confirmed.	A2, A11

4. `total_amount` — zero/negative structural boundary [A2, A11]

Channel: Direct API. Same fixture context as above.

ID	<code>total_amount</code>	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-07	-1	LB-1	Rejected or recalculated to 58,000,000	Executed: 200, <code>orderId:6</code> , <code>total_amount:-1</code> persisted as-is (confirmed via GET). Bug confirmed: SQLite <code>INTEGER</code> column has no constraint; order recorded with a negative total	A2, A11
TC-08	0	LB	Rejected or recalculated to 58,000,000	Executed: 200, <code>orderId:7</code> , <code>total_amount:0</code> persisted as-is (confirmed via GET). Bug confirmed: a 58-million-VND cart recorded as free	A2, A11

ID	<code>total_amount</code>	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-09	1	LB+1	Rejected or recalculated to 58,000,000	Executed: 200, <code>orderId:8</code> , <code>total_amount:1</code> persisted as-is (confirmed via GET). Bug confirmed: near-total elimination of amount owed	A2, A11

5. `total_amount` — upper business-cap boundary [A3, A11]

Channel: Direct API. Same fixture context as above.

ID	<code>total_amount</code>	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-10	999999999	UB-1	Recalculated to 58,000,000	Executed: 200, <code>orderId:9</code> , persisted verbatim as 999,999,999 (confirmed via GET). Bug confirmed.	A3, A11
TC-11	1000000000	UB	Recalculated to 58,000,000	Executed: 200, <code>orderId:10</code> , persisted verbatim as 1,000,000,000 (confirmed via GET). Bug confirmed.	A3, A11
TC-12	1000000001	UB+1	Recalculated to 58,000,000	Executed: 200, <code>orderId:11</code> , persisted verbatim as 1,000,000,001 (confirmed via GET). Bug confirmed.	A3, A11

6. `total_amount` — architectural-extreme probe (added-value, beyond core BVA) [A4]

ID	<code>total_amount</code>	Expected	Actual / Bug flag	Assumptions
TC-13	9007199254740991 (<code>Number.MAX_SAFE_INTEGER</code>)	N/A — exploratory probe only	Executed: 200, <code>orderId:12</code> ; GET <code>/api/orders/my-orders</code> confirmed <code>total_amount</code> persisted exactly as 9007199254740991 — no precision loss, no overflow/coercion anomaly. Confirms the prediction exactly: persists verbatim like every other tampered value	A4

7. `total_amount` — non-ordered EP classes (malformed / omitted) [A2, A11]

ID	<code>total_amount</code>	Class	Expected (per spec)	Actual / Bug flag	Assumptions
TC-14	<code>58000000.5</code> (decimal)	I5 — malformed format	Rejected or recalculated to 58,000,000	Executed: 200, <code>orderId:13</code> ; GET confirmed <code>total_amount</code> persisted as <code>58000000.5</code> (kept as REAL, not coerced to integer) — confirms the prediction exactly	A2, A11
TC-15	Field omitted entirely from JSON body	I6 — missing required field	Rejected (400) or recalculated to 58,000,000	Executed: 200, <code>orderId:14</code> , no crash; GET confirmed <code>total_amount: null</code> . Confirms the prediction exactly — same <code>undefined→NULL</code> coercion as FR-04's TC-08/TC-10/TC-22	A11

8. Cart emptiness — isolated + stacked interaction [A5]

Channel: Direct API. Valid token; no coupon; valid address; `items: []`.

ID	Cart	<code>total_amount</code>	Expected (per spec)	Actual / Bug flag	Assumptions
TC-16	Empty (0 items)	<code>0</code>	Rejected — nothing to purchase	Executed (before the fixture cart was built, cart confirmed empty via <code>GET /api/cart</code>): 200, <code>orderId:1</code> created with <code>total_amount:0</code> . Bug confirmed (per scoping decision): backend never reads or validates <code>items</code> at all (<code>Checkout.jsx</code> sends <code>items: cart</code> , but <code>server.js:299</code> never destructures it) — an order is created regardless of cart size	A5
TC-17 (deliberate interaction)	Empty (0 items)	<code>1000000000</code>	Rejected — nothing to purchase, and even if allowed, recalculated total should be 0	Executed (empty cart, immediately after TC-16): 200, <code>orderId:2</code> created with <code>total_amount:1000000000</code> persisted verbatim (confirmed via GET). Bug confirmed (severity-stacked): a fully attacker-controlled total is	A5, A2

ID	Cart	<code>total_amount</code>	Expected (per spec)	Actual / Bug flag	Assumptions
				accepted for an order with zero items — the two confirmed defects (no cart-size guard + no total recalculation) compound into a "phantom paid order" for literally nothing in the cart	

9. Coupon — binary smoke test (deliberate interaction) [A6]

Channel: Direct API. Valid token; fixture cart; valid address; assume one valid, active, pre-existing coupon with a 10% discount, so `POST /api/apply-coupon` returns `final_amount = 52200000`.

ID	Coupon state	<code>total_amount submitted</code>	Expected	Actual / Bug flag	Assumptions
TC-18	Applied (<code>coupon_id</code> attached)	Whatever value <code>/api/apply-coupon</code> returns as <code>final_amount</code>	Checkout persists correctly with whatever the backend legitimately recalculates net of a validly-applied discount (coupon math itself out of scope)	Executed: obtained a coupon-computed <code>total_amount</code> via <code>POST /api/apply-coupon (code:"SAVE10")</code> , then submitted checkout with that value and <code>coupon_id</code> attached → 200, <code>orderId:15</code> , persisted verbatim (confirmed via GET); <code>POST /api/coupon-usage</code> succeeded. No new FR-08 defect beyond #1/#3 (total-trust, cart-not-cleared) — the coupon path still just stores whatever value arrives. (Note: the coupon-computed value did not match this case's originally assumed figure; the discrepancy is coupon-calculation logic owned by FR-09, out of scope for FR-08 per A6.)	A6

10. `shipping_address` — lightweight confirmatory case [A7]

Channel: Direct API. Valid token; fixture cart; `total_amount = 58000000`; no coupon.

ID	shipping_address	Expected	Actual / Bug flag	Assumptions
TC-19	"" (empty string)	Spec is silent; reasonable expectation is a non-empty address is required for delivery	Executed: 200, <code>orderId:16</code> , <code>shipping_address:""</code> persisted as-is (confirmed via GET). No validation observed in the <code>/api/checkout</code> handler. Documented as a minor observation, not scored as a headline FR-08 defect (overlaps FR-04's own address boundary analysis territory)	A7

11. Order owner identity — confirmatory regression guard [A9]

Channel: Direct API. User A holds a valid token; request body additionally includes a forged `user_id` field pointing at User B, alongside an otherwise-honest fixture cart/total.

ID	Scenario	Expected	Actual / Bug flag
TC-20	Forged <code>user_id</code> in body, valid token for User A	Persisted <code>orders.user_id =</code> User A (token owner), regardless of any client-supplied owner field	Executed: submitted <code>user_id:1</code> (Admin) in the body while authenticated as the test user (id 2). 200, <code>orderId:17</code> ; GET confirmed <code>orders.user_id = 2</code> (token owner), not 1; Admin's own GET <code>/api/orders/my-orders</code> remained empty (no leaked/forged order). Matches — no defect. <code>userId</code> is bound only to <code>req.user.id</code> from the verified JWT (<code>server.js:298</code>); the body destructure (<code>{ total_amount, shipping_address }</code>) never reads a <code>user_id</code> /owner-like field at all, so a forged value is silently ignored. Structural guarantee confirmed, same treatment as FR-04's cross-user regression guard

Summary of Confirmed/Expected Defects

- Critical: backend never recalculates `total_amount` from server-side cart/product data.** `POST /api/checkout` (`server.js:297-309`) inserts the client-submitted `total_amount` verbatim, with no product-price lookup at all — the exact scenario the spec explicitly prohibits (TC-05–TC-15, TC-17). **GitHub Issue:** [BUG-B-08 #38](#)
- Critical: the checkout total is directly user-editable in the UI.** `Checkout.jsx:110-121` renders a plain `<input type="number">` bound to `editableTotal`, the literal opposite of "không cho phép người dùng chỉnh sửa trực tiếp." Combined with defect #1, any user can pay any amount for any cart (TC-01, TC-05–TC-15). **GitHub Issue:** [BUG-B-07 #37](#)
- Cart is never cleared after a successful checkout.** `clearCart()` (defined in `CartContext.jsx:18`) is never invoked in `Checkout.jsx`'s `handleCheckout` — confirmed even on the nominal happy path (TC-01). **GitHub Issue:** [BUG-B-09 #39](#)
- No cart-size guard at checkout.** The backend never reads or validates the `items` array; an order is created for an empty cart (TC-16), and this stacks with defect #1 to allow a fully attacker-priced "phantom" order for zero products (TC-17). **GitHub Issue:** [BUG-B-10 #40](#)

5. **No persisted record of purchased line items.** The `orders` table (`database.js:74-81`) has no `order_items`/line-item table; the `items` array sent by the frontend is silently dropped by the backend. Out of scope for FR-08 test cases per explicit decision. Note: on review of FR-11's actual spec, this gap does **not** block FR-11 compliance — FR-11 only requires displaying order code, date, total amount, and status, none of which depend on itemized line items.
6. **No defect (corrected):** a logged-out user can load the `/checkout` page directly by URL, but any actual payment attempt still correctly fails at the API (401, per TC-02). The spec's actual requirement ("only logged-in users may proceed to checkout/payment") concerns the checkout action, not page-view access, and is satisfied. This was originally over-flagged as a bug based on an inferred reading beyond the literal spec text; corrected after manual verification (TC-04).
7. **No confirmed defect** in: auth-state gating (TC-02, TC-03), order-owner identity / cross-user isolation (TC-20), or the coupon-applied path introducing any *additional* breakage beyond defects #1 and #3 (TC-18).
8. **Confirmed: SQLite type-affinity and driver NULL-coercion behavior at the `total_amount` boundaries.** A decimal value (TC-14, `58000000.5`) is kept as REAL and persisted as-is, uncoerced to integer. An omitted `total_amount` field (TC-15) is bound as `undefined`, which the Node sqlite3 driver coerces to SQL `NULL` — the order is created with `total_amount: null` rather than being rejected, consistent with FR-04's analogous omitted-field findings.

FR-19 — User Management (Admin): Domain Testing / BVA Test Case Suite

Methodology: Domain Testing via Equivalence Partitioning (EP) and Boundary Value Analysis (BVA), following a 6-step human-in-the-loop QA process. Combinatorial strategy: **Isolated Boundaries + Happy-Path Interactions** (Option C), with two deliberate UI-channel confirmatory cases beyond the core API-driven matrix. 15 test cases total.

Source spec (FR-19, Vietnamese, instructor-provided):

Admin xem danh sách tất cả người dùng (không lộ mật khẩu). Admin có thể xóa người dùng, ngoại trừ không được xóa chính tài khoản đang đăng nhập.

Code under test:

- Frontend: `frontend-admin/src/App.jsx` (Users tab render, `fetchData`, `deleteUser`, login-time role gate)
- Backend: `backend/server.js` (GET `/api/admin/users`, DELETE `/api/admin/users/:id`, `authenticateToken`, POST `/api/login` JWT signing)
- DB schema: `backend/database.js` (`users` table — `role` TEXT DEFAULT 'user', no CHECK constraint; `id` INTEGER PRIMARY KEY AUTOINCREMENT)

Preconditions (unless a case states otherwise): Actor holds a valid, signature-verifiable JWT (V6 assumption — authentication state itself, e.g. missing/expired tokens, is out of scope for FR-19). The specific `id/role` claims encoded in that token are the variable under test per case.

Fixture (used throughout, drawn from actual `database.js` seed data):

id	name	email	role
1	Admin User	admin@eshop.com	admin
2	Test User	test@eshop.com	user

Assumption Legend

Tag	Assumption
A1	Fixture seed rows: id=1 "Admin User" (role=admin), id=2 "Test User" (role=user), per <code>database.js:91-93</code> — used throughout as concrete self/other/target identities — Step 0/3
A2	Obtaining a caller token with an arbitrary role (e.g. " <code>superadmin</code> ", EC3) requires first persisting that string to some user's DB <code>role</code> column (unconstrained TEXT, no <code>CHECK</code> constraint) and then performing a normal login — the login handler signs whatever value is present into the JWT (<code>server.js:51</code>) — Step 2
A3	V3's BVA is computed against the <code>id</code> column's structural domain (positive integer, <code>AUTOINCREMENT</code> floor = 1), independent of any specific row's existence — Step 3

Tag	Assumption
A4	BVA's "valid, existing id" boundary points (LB=1, LB+1=2) collapse onto the already-planned self (EC5) / other (EC4) happy-path cases (TC-07 / TC-06 respectively), since the 2-row seed fixture assigns id=1 to the admin/self account and id=2 to the only available "other" account — no additional dedicated boundary cases needed — Step 4/5 combinatorial reduction
A5	id-format invalid classes (0, -1, "abc", 999999) are each tested in isolation under a fixed valid admin caller role, independent of the self/other dimension, per the invalid-class isolation rule and the Step 4 finding that V2 only applies to V3=EC6 — Step 4
A6	Role-invalid isolated DELETE cases (TC-12/13) deliberately target the seeded Admin account (id=1) as the "other, existing" victim, rather than an arbitrary row, to demonstrate maximal real-world impact of the missing role check — Step 4/5
A7	GET's role-invalid isolated cases (TC-04/05) are fixed at table size N=2 (the normal seeded state), since table size does not interact with the role-rejection outcome — Step 4
A8	Password-field exclusion (V5) is checked as a standing oracle on every GET case in this suite, not a separately partitioned variable — Step 2
A9	TC-07, TC-12, and TC-13 are each destructive to the same seeded Admin row (id=1); each must be executed against an independently reseeded DB state, never sequentially against shared, unreset fixture data — Step 5 execution note
A10	Non-admin-role cases (TC-04, TC-05, TC-12, TC-13, TC-15) require obtaining a valid JWT for a non-admin account via the shared POST /api/login endpoint (or the regular web frontend), since the admin panel's own login form blocks non-admin logins client-side (App.jsx:65-68) — this is a UI convenience gate only, not a security control, since the backend never checks role itself — Step 0/4
A11	TC-14/TC-15 (UI-channel cases) are added-value scenarios beyond the core EP/BVA methodology, included to demonstrate F1/F2 are trivially triggerable through the real admin UI, not just crafted API calls — mirrors the FR-04/FR-08 reports' treatment of bonus channel-crossing cases

1. GET /api/admin/users — Happy-path × table size (V4 full cross, role=admin)

Channel: Direct API. Caller: role=**admin** (EC1) [A1].

ID	Table size	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-01	0 users (EC10)	LB	200, [], no crash	Executed (table emptied via TC-06+TC-07 deleting both seeded rows): 200, [] returned, no crash. Matches — db.all returns [] naturally; no branching to fail on	A8
TC-02	1 user (EC11)	LB+1	200, array of 1, no	Executed (table shrunk to 1 row via TC-06): 200, array of 1 (Admin only), no	A8

ID	Table size	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
			<code>password</code> field	<code>password</code> field present. Matches — <code>SELECT</code> explicitly excludes <code>password</code>	
TC-03	2 users (EC12)	—	200, array of 2, no <code>password</code> field	Executed on the freshly reseeded 2-row fixture: 200, array of 2, no <code>password</code> field present. Matches	A1, A8

2. GET /api/admin/users — Role isolated invalid classes (F1)

Channel: Direct API. Table size = 2 (fixed, A7).

ID	Caller role	Expected (per spec)	Actual / Bug flag	Assumptions
TC-04	<code>user</code> (EC2)	403 — rejected, no data	Executed: 200, identical full 2-row list (both users, no password field) returned to a non-admin (<code>role:"user"</code>) caller. Bug confirmed (F1) : no role gate exists	A7, A10
TC-05	<code>"superadmin"</code> (EC3, garbage)	403 — rejected	Executed: with a freshly-minted JWT carrying <code>role:"superadmin"</code> (obtained by exploiting the FR-04 role-escalation bug then re-logging-in), 200, identical full list returned. Bug confirmed (F1) : confirms the missing check is total, not merely "unrecognized real roles slip through"	A2, A7, A10

3. DELETE /api/admin/users/:id — Happy-path × self/other (V2 full cross, role=admin)

Channel: Direct API. Caller: role=`admin` (EC1), id=1.

ID	Target	Class	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-06	id=2 (Test User)	EC4 — other	LB+1	200, Test User row removed	Executed: 200 "User deleted"; confirmed via subsequent GET that only Admin (id=1) remained. Matches — legitimate admin action succeeds correctly	A1, A4
TC-07	id=1 (own account)	EC5 — self	LB	400/403 — rejected, account not deleted	Executed (on a freshly reset DB, per destructive-test protocol; DB reset again immediately after): 200 "User deleted"; confirmed via subsequent GET (using the still signature-valid admin token)	A1, A4, A9

ID	Target	Class	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
					that the users table was left empty. Bug confirmed (F2, critical): the admin's own row is removed; the calling account ceases to exist	

4. DELETE /api/admin/users/:id — Target-id isolated invalid classes (F4)

Channel: Direct API. Caller: role=**admin** (fixed, A5).

ID	id value	Class	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-08	0	EC-invalid	LB-1	400/404 — rejected	Executed: 200 "User deleted" ; confirmed via subsequent GET that both real rows (id=1, id=2) remained untouched. Bug confirmed (F4): WHERE id = 0 matches 0 rows; handler ignores this.changes ; false-positive success	A3, A5
TC-09	-1	EC8	—	400/404 — rejected	Executed: 200 "User deleted" , 0 rows actually affected (both real rows confirmed intact afterward). Bug confirmed (F4): same false-positive 200	A3, A5
TC-10	"abc"	EC9	—	400 — rejected (wrong type)	Executed: 200 "User deleted" , 0 rows actually affected (both real rows confirmed intact afterward). Bug confirmed (F4): resolves the prior prediction exactly — SQLite's NUMERIC-affinity comparison fails to match any row for the non-numeric string, this.changes is ignored, and the handler still reports false-positive success	A5
TC-11	999999	EC7	—	404 — rejected (not found)	Executed: 200 "User deleted" , 0 rows actually affected (both real rows confirmed intact afterward). Bug confirmed (F4): same false-positive 200	A5

5. DELETE /api/admin/users/:id — Role isolated invalid classes (F1, compounded)

Channel: Direct API. Target: id=1 (Admin account — deliberately chosen victim, A6).

ID	Caller role	Expected (per spec)	Actual / Bug flag	Assumptions
TC-12	<code>user</code> (EC2)	403 — rejected, Admin account untouched	Executed (on a freshly reset DB immediately before, and reset again immediately after, per destructive-test protocol): <code>DELETE /api/admin/users/1</code> using the plain <code>role:"user"</code> token → 200 "User deleted"; confirmed via GET that only Test User (id=2) remained — Admin account gone. Bug confirmed (F1, critical, compounded): total loss of the only admin account in the system, performed by a plain non-admin user	A1, A6, A9, A10
TC-13	<code>"superadmin"</code> (EC3, garbage)	403 — rejected	Executed (on a freshly reset DB immediately before, and reset again immediately after, per destructive-test protocol): <code>DELETE /api/admin/users/1</code> using the <code>role:"superadmin"</code> garbage-role token → 200 "User deleted"; confirmed via GET that only Test User (id=2) remained. Bug confirmed (F1): confirms the missing check holds regardless of what nonsense role string is presented	A2, A6, A9, A10

6. UI-channel confirmatory cases (added-value, beyond core EP/BVA methodology) [A11]

ID	Scenario	Expected (per spec)	Actual / Bug flag	Assumptions
TC-14	Admin (role=admin) logs into <code>frontend-admin</code> normally, opens "Quản lý Người dùng", clicks "Xóa" on the row matching their own account	Action blocked or requires explicit confirmation before any request is sent	Executed (manual UI test): clicked "Xóa" on the Admin's own row — no confirmation dialog appeared at all, and the account was deleted immediately. Bug confirmed (F2): <code>deleteUser(u.id)</code> (<code>App.jsx:76-83</code>) fires unconditionally for any row, with no confirmation dialog and no self-id awareness — the app never decodes its own id	A9, A11

ID	Scenario	Expected (per spec)	Actual / Bug flag	Assumptions
			(confirmed in Step 0). The DELETE succeeds exactly as in TC-07; the admin is locked out on the next request	
TC-15	A non-admin token (obtained via the shared <code>POST /api/login</code> , e.g. Test User's credentials) is manually placed into <code>localStorage["adminToken"]</code> , and the admin panel is reloaded — bypassing the login form's client-side role check (<code>App.jsx:65-68</code>) entirely	Panel refuses to render / redirects a non-admin session	Executed (manual UI test): placed a non-admin token into <code>localStorage["adminToken"]</code> and reloaded the admin panel — the full dashboard rendered anyway. Bug confirmed (F1): <code>useEffect</code> (<code>App.jsx:34-39</code>) calls <code>fetchData()</code> for any truthy <code>token</code> regardless of role, including a working Users tab with delete buttons — same root cause as TC-04, now demonstrated through the real UI rather than a raw API call	A10, A11

Summary of Confirmed/Expected Defects

- Critical: no role check on either admin/users endpoint (F1).** `GET /api/admin/users` and `DELETE /api/admin/users/:id` apply only `authenticateToken`, which verifies the JWT signature but never inspects `req.user.role`. Any authenticated account — a real `user`-role account or one carrying an arbitrary garbage role string — can list all users and delete any user, including the sole admin account (TC-04, TC-05, TC-12, TC-13). Confirmed reachable through the real UI, not just direct API calls (TC-15). **GitHub Issue:** [BUG-C-08 #41](#)
- Critical: no self-deletion guard (F2).** The DELETE handler never compares `req.params.id` to `req.user.id`; an admin can delete their own currently-logged-in account, directly violating the spec's explicit exception clause (TC-07). Confirmed trivially triggerable via a normal UI click, with no confirmation dialog (TC-14). **GitHub Issue:** [BUG-C-09 #42](#)
- Compounded severity:** F1 and F2 together mean a plain `user`-role account can delete the system's only admin account (TC-12) — a full administrative lockout, not merely an unauthorized read/delete of an arbitrary row.
- Minor: DELETE never validates rows affected (F4).** The handler ignores `this.changes`; deleting a non-existent (999999), negative (-1), zero (0), or non-numeric ("abc") id all confirmed to return a false-positive 200 "User deleted" with zero actual effect (TC-08, TC-09, TC-10, TC-11) — including the non-numeric case, confirmed via execution to behave identically via SQLite's type-affinity fallback. **GitHub Issue:** [BUG-C-10 #43](#)
- No defect:** password is never exposed in any GET response, across all table sizes and even in the erroneous non-admin-accessible responses (TC-01–TC-05).
- No defect:** an admin deleting a different, existing user works correctly and is the only scenario in this suite that behaves per spec (TC-06).

FR-11 — Order History View (User), Mobile App: Domain Testing / BVA Test Case Suite

Methodology: Domain Testing via Equivalence Partitioning (EP) and Boundary Value Analysis (BVA), following a 6-step human-in-the-loop QA process. Combinatorial strategy: **Isolated Boundaries + Happy-Path Interactions** (Option C), with two deliberate targeted interactions (a combined status × order-count × cancel-button happy path, and a single representative auth state used to prove `/orders/:id` is wide open rather than crossing every auth state against every ownership/existence combination). 11 test cases total.

Source spec (FR-11, Vietnamese, instructor-provided):

Người dùng chỉ xem được đơn hàng của chính mình. Hiển thị: Mã đơn, Ngày đặt, Tổng tiền, Trạng thái hiện tại. Trạng thái phải được dịch sang tiếng Việt rõ ràng và phân biệt màu sắc.

Scope note: View-only. Order cancellation and admin-side state transitions belong to FR-10/FR-18; the cancel button appearing on this screen is checked only as a single lightweight binary presence/absence oracle (V10), not a state-machine sweep.

Code under test:

- Frontend: `frontend-mobile/App.js` (`fetchOrders`, `statusLabel`, order-history render block, cancel button)
- Backend routes: `backend/server.js` (`GET /api/orders/my-orders`, `GET /api/orders/:id`, `PUT /api/admin/orders/:id/status`, `authenticateToken`)
- DB schema: `backend/database.js` (`orders` table)

Preconditions (unless a case states otherwise): Actor "User A" is logged in with a valid JWT. A second, distinct "User B" account exists for ownership-mismatch cases.

Fixture 1 — existence/ownership boundary (used in Sections 3–4): 3 orders created sequentially via checkout, relying on the `orders.id AUTOINCREMENT` column (table starts empty; no order seed data in `database.js`):

id (creation order)	Owner
1	User A
2	User B (different user)
3	User A

Fixture 2 — status/count happy path (used in Section 2): User A holds 5 orders, one per known status value, set via `PUT /api/admin/orders/:id/status`:

Status (DB value)	Vietnamese label (expected)
pending	Chờ xác nhận
confirmed	Đã xác nhận

Status (DB value)	Vietnamese label (expected)
shipping	Đang giao
delivered	Đã giao
canceled	Đã hủy

Assumption Legend

Tag	Assumption
A1	Fixture 1 — 3 sequentially-created orders, ids 1/2/3, with id=2 belonging to a different user (User B) — used for both the existence boundary (V4) and ownership/IDOR (V3) test cases, since ids are globally auto-incremented (not per-user) — Step 3 sign-off
A2	Fixture 2 — User A holds exactly 5 orders, one per known status value, reachable only via PUT /api/admin/orders/:id/status 's hardcoded transition table, traced in Step 0 to guarantee no out-of-enum value can ever be written — used for the combined V8×V9×V10 happy path (TC-03) — Step 4 dependency #3
A3	Out-of-enum status values are excluded entirely from this suite; no live write path (checkout hardcodes "pending" , cancel only writes "canceled" , admin update validates against a fixed transition table) can ever produce one — Step 0 decision #3
A4	"Mã đơn" is treated as the raw autoincrement id ; no dedicated order-code field exists in the schema — Step 0 decision #4
A5	"Ngày đặt" (order date, locale/timezone formatting) is excluded from formal BVA as a display concern, verified only passively within happy-path cases — Step 0 decision #7
A6	"Tổng tiền" business-rule validity (the value itself) is out of scope for FR-11 and owned by FR-08's checkout logic; FR-11 only verifies correct display of whatever value is already persisted — Step 1, V7
A7	Number-of-orders (V9) receives EP-only treatment (0/1/many); no pagination exists anywhere in the code, so no upper BVA boundary applies — Step 2/3 sign-off
A8	Cancel-button visibility (V10) is checked only as a single binary presence/absence oracle, bundled into TC-03, not swept against the full order-state-machine transition logic (owned by FR-10/FR-18) — Step 0 scope note
A9	Expired and malformed/invalid JWTs are not tested as separate classes — jwt.verify 's error callback (server.js:105-107) handles both identically (403), with no distinguishing logic to split on — Step 2 partition rule (classes handled identically are not split)
A10	A single representative unauthenticated caller state is used for the IDOR probe (TC-10), rather than crossing every auth state against V3/V4 on /orders/:id , since that endpoint has no auth middleware at all and every auth state is provably equivalent there — Step 4 dependency #4
A11	"Mã đơn" (id) and "Tổng tiền" (total_amount) display-correctness are verified as passive oracles bundled into the happy-path cases (TC-01–TC-03, TC-07, TC-08), not given dedicated sections of

Tag	Assumption
	their own, since both are read-only fields with no independently testable invalid class for this FR — Step 1, V5/V7
A12	A non-numeric id (e.g. <code>/api/orders/abc</code>) does not get its own test case — it is predicted to collapse into the same does-not-exist / 404 class as TC-06/TC-09, via SQLite's type-affinity fallback when comparing a TEXT parameter against an INTEGER-affinity <code>id</code> column. This is a database-engine-behavior prediction, not a directly-observed code fact (the same reasoning class as FR-19's TC-10), and remains unconfirmed pending actual test execution — Step 3

1. GET /my-orders — Order-count boundary, isolated (V9)

Channel: Direct API / Mobile UI (identical call path — `fetchOrders`, `App.js:174`). Caller: User A, valid token.

ID	Order count	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-01	0 orders	LB	"Bạn chưa có đơn hàng nào." empty-state message, no crash	Executed: <code>GET /api/orders/my-orders</code> as User A on a fresh, order-free account → 200, <code>[]</code> . Matches — <code>orders.length === 0</code> branch renders the empty-state text (<code>App.js:947-948</code>)	A7
TC-02	1 order	LB+1	Single order card renders with Mã đơn/Ngày đặt/Tổng tiền/Trạng thái	Executed: after one checkout, <code>GET /api/orders/my-orders</code> as User A → 200, array of exactly 1 order (id, total_amount, status, shipping_address, created_at all present and correct). Matches — <code>orders.map</code> renders one <code>orderCard</code> with all four fields (<code>App.js:950-960</code>)	A7, A11

2. GET /my-orders — Status × count × cancel-button combined happy path (V8, V9=many, V10) [A2, A8]

Channel: Direct API / Mobile UI. Caller: User A, valid token. Fixture 2 (5 orders, one per status).

ID	Scenario	Expected (per spec)	Actual / Bug flag	Assumptions
TC-03	Single fetch/render of all 5 orders (pending, confirmed, shipping, delivered, canceled)	All 5 statuses render translated Vietnamese labels (per Fixture 2 table); cancel button visible only on pending/confirmed rows, absent on the other 3	Executed (Direct API leg): built Fixture 2 by giving User A 5 orders (ids 1,3,4,5,6) and driving each through <code>PUT /api/admin/orders/:id/status</code> to a distinct terminal status. A single <code>GET /api/orders/my-orders</code> call returned all 5 orders in one response with status values exactly pending (id1), confirmed (id3), shipping (id4), delivered (id5),	A2, A8, A11

ID	Scenario	Expected (per spec)	Actual / Bug flag	Assumptions
			canceled (id6) — matches Fixture 2 exactly, confirming the backend data underlying this case is correct. The Vietnamese-label translation and cancel-button gating are frontend rendering logic (App.js:331-339, App.js:961) not exercisable via curl — those conclusions still stand as originally assessed by code reading, not independently re-verified in a browser this session. No status row is visually differentiated by color (see TC-11) — all 5 render in identical default text styling despite carrying different, correctly-translated labels	

3. GET /my-orders — Authentication isolated invalid classes (V2) [A9]

Channel: Direct API. Otherwise-honest request (User A's identity implied by token where valid).

ID	Auth condition	Expected (per spec)	Actual / Bug flag	Assumptions
TC-04	Missing Authorization header	401; no data returned	Executed: 401 {"error": "Unauthorized"}, no data. Matches — authenticateToken (server.js:103) returns 401 before the query runs	—
TC-05	Invalid/malformed or expired token	Rejected; no data returned	Executed: malformed token (Bearer invalid.token.value) → 403 {"error": "Forbidden"}, no data. Matches — jwt.verify err branch (server.js:106) returns 403; expired and malformed tokens are indistinguishable at this check and both correctly rejected	A9

4. GET /orders/:id — Existence boundary (V4), requester = owner or n/a [A1]

Channel: Direct API (endpoint is never called by the mobile UI itself, per Step 0 finding). Caller: User A, valid token (irrelevant to outcome — see Section 5).

ID	id	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
TC-06	0	LB-1	Does not exist → 404	Executed: 404 {"error": "Order not found"}. Matches — if (!order) return	A1

ID	id	Boundary	Expected (per spec)	Actual / Bug flag	Assumptions
				<code>res.status(404)... (server.js:346);</code> id 0 was never assigned by <code>AUTOINCREMENT</code>	
TC-07	1	LB	Exists, User A's own order → 200 with correct fields	Executed: 200, order returned with all fields (<code>id</code> , <code>total_amount</code> , <code>status</code> , <code>shipping_address</code> , <code>created_at</code>) correct and matching Fixture 1. Matches	A1, A11
TC-08	3	UB	Exists, User A's own order → 200 with correct fields	Executed: 200, order returned with all fields correct and matching Fixture 1. Matches	A1, A11
TC-09	4	UB+1	Does not exist → 404	Executed (run before Fixture 2 created any further orders, preserving the boundary): 404 <code>{"error":"Order not found"}</code> . Matches — first never-created id at that point, same 404 path as TC-06	A1

Note (no dedicated test case, per A12): a non-numeric id (e.g. `id="abc"`) is predicted, not confirmed, to fall into this same does-not-exist / 404 class via SQLite's type-affinity fallback — see A12 for why this is treated as a prediction rather than a settled fact.

5. GET /orders/:id — Ownership / IDOR (V3), critical finding [A1, A10]

Channel: Direct API — **not reachable through mobile UI navigation** (the app only ever calls `/my-orders`, per Step 0), but directly reachable by any HTTP client regardless of UI usage.

ID	id	Caller	Expected (per spec)	Actual / Bug flag	Assumptions
TC-10	2 (User B's order — same row as Fixture 1's LB+1/UB-1 boundary value)	No <code>Authorization</code> header at all (representative single auth state, A10)	Spec: "Người dùng chỉ xem được đơn hàng của chính mình" → rejected (401/403/404), not User B's data	Executed: <code>GET /api/orders/2</code> with no <code>Authorization</code> header at all → 200 , full JSON body of User B's order (<code>id:2</code> , <code>user_id:3</code> , <code>total_amount:5000000</code> , <code>status:"pending"</code> , <code>shipping_address:"456 Le Loi, Q1, HCMC"</code>) returned to the completely unauthenticated caller. Critical bug confirmed (IDOR): <code>GET /api/orders/:id (server.js:344-349)</code> has no	A1, A10

ID	id	Caller	Expected (per spec)	Actual / Bug flag	Assumptions
				<code>authenticateToken</code> middleware and no <code>user_id</code> filter (<code>SELECT * FROM</code> <code>orders WHERE id = ?</code>). Confirms the spec's core ownership rule is entirely unenforced on this endpoint, in direct contrast to <code>/my-</code> <code>orders</code> 's correct <code>WHERE</code> <code>user_id = req.user.id</code> filter (<code>server.js:313</code>)	

6. Color differentiation — single confirmed defect (V11)

Channel: Mobile UI (visual inspection of rendered screen; no API involved).

ID	Scenario	Expected (per spec)	Actual / Bug flag	Assumptions
TC-11	Rendered order- history screen, any/all of Fixture 2's 5 statuses (reuses TC- 03's render)	Spec: "phân biệt màu sắc" — status text must be visually color- differentiated per status	Confirmed bug: full-file grep for <code>color/Color</code> across <code>App.js</code> shows no <code>statusColor/getStatusColor</code> helper and no conditional styling keyed by status anywhere; the status <code><Text></code> (<code>App.js:960</code>) carries no <code>style</code> prop at all. All 5 statuses render in identical, undifferentiated default text color. Absence is total, not partial — one case suffices, not a per-status sweep	—

Summary of Confirmed/Expected Defects

- Critical: no authentication or ownership check on `GET /api/orders/:id` (IDOR).** The endpoint (`server.js:344-349`) has no `authenticateToken` middleware and its query has no `user_id` filter — any caller, authenticated or not, can fetch any order by id and see another user's `total_amount`, `status`, and `shipping_address` (TC-10). This directly contradicts the spec's first line ("chỉ xem được đơn hàng của chính mình") and stands in stark contrast to `/my-orders`, which correctly scopes to the caller (TC-01–TC-05). Not reachable via the mobile UI's own navigation, but directly reachable by any HTTP client. **GitHub Issue:** [BUG-D-10 #44](#)
- Confirmed: no color differentiation of order status anywhere in the mobile app.** No status-keyed styling exists in `frontend-mobile/App.js`; all 5 statuses render in identical default text color (TC-11), contradicting the spec's explicit "phân biệt màu sắc" requirement. Status *translation* to Vietnamese, by contrast, is fully correct (TC-03). **GitHub Issue:** [BUG-D-11 #45](#)
- No defect:** `/my-orders` correctly requires a valid token and correctly scopes results to the caller's own orders across all order-count classes (0/1/many) (TC-01–TC-05).

4. **No defect:** all 5 known status values translate to the correct Vietnamese label, and the cancel button is correctly gated to appear only on **pending/confirmed** orders (TC-03).
5. **No defect:** **GET /orders/:id**'s existence-boundary handling itself is correct — ids 0 and 4 (never assigned) correctly 404, ids 1 and 3 (existing) correctly return data (TC-06–TC-09). The endpoint's failure is exclusively the missing ownership/auth check (defect #1), not its existence-check logic.
6. **Out of scope, confirmed non-issue for FR-11:** the **orders** table's lack of **order_items**/line-item persistence (flagged in FR-08's report) does not block FR-11 — none of the four required display fields (Mã đơn, Ngày đặt, Tổng tiền, Trạng thái) depend on itemized products.